

NobleProg

The World's Local Training Provider



Sesión 3

Domain-Driven Design y descomposición de datos

Migración de Monolitos a Microservicios

Carlos Georg Lübbe · 3 h

Objetivo de hoy

Encontrar **buenos límites** (DDD) y abordar lo más difícil: **los datos**.

1. **DDD**: lenguaje ubicuo y *bounded contexts* — ¿por dónde corto?
2. **Encontrar los límites**: context maps, core/supporting/generic, granularidad
3. **Descomposición de datos**: BD por servicio, partir la BD compartida
4. **Consistencia sin transacciones distribuidas**: eventual, sagas, outbox

1. Domain-Driven Design

¿Por qué DDD y microservicios?

La pregunta más cara de la migración: **¿por dónde corto?** Un corte malo = monolito distribuido (Sesión 1).

DDD (Eric Evans, 2003) alinea los límites del software con los del **negocio** — los límites más estables que existen.

- **DDD estratégico** — *el mapa: ENTRE contextos, dónde están los límites* (bounded contexts, lenguaje ubicuo, context maps). ← **esto necesita la migración**
- **DDD táctico** — *el interior: DENTRO de un contexto, cómo se construye* (entidades, agregados...). Solo tocaremos *agregados*.

Lenguaje ubicuo

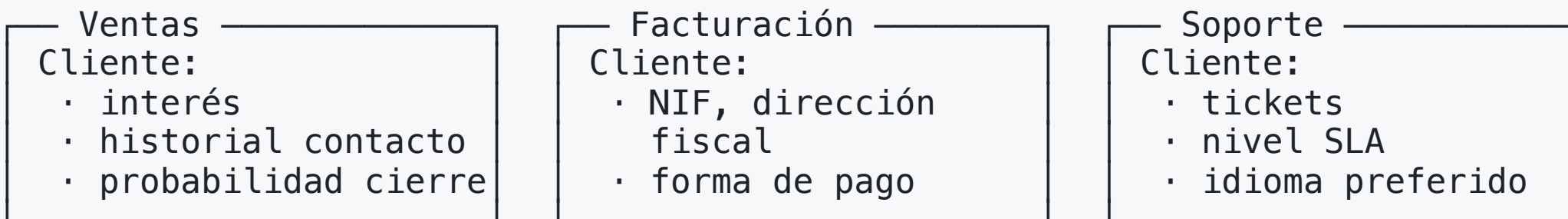
Vocabulario **compartido** entre negocio y desarrollo, usado igual en conversaciones, documentos y **código**. Si negocio dice "póliza" y el código dice `ContractRecord`, cada conversación necesita traducción.

La observación que encuentra límites: el lenguaje **cambia de significado según el contexto**.

"Cliente" no es lo mismo para **ventas** (un lead), **facturación** (un titular fiscal) y **soporte** (alguien con tickets). No es mala comunicación: son **conceptos distintos** que comparten nombre.

Bounded contexts

Frontera dentro de la cual un **modelo y su lenguaje** son válidos y consistentes. No hace falta un **modelo único de empresa** (el "Cliente canónico" de 200 campos que no sirve a nadie):



mismo nombre, tres modelos distintos – y está bien

Cómo detectar fronteras en la práctica

- **Cambios de significado** de una palabra según el departamento
- **Sinónimos**: dos áreas llaman distinto a lo mismo
- Fronteras **organizativas** y de proceso de negocio
- En el monolito: grupos de tablas/módulos que **cambian juntos**; "traducciones" implícitas (mappers, columnas reinterpretadas)
- Talleres tipo **EventStorming**: negocio y desarrollo mapean los eventos del dominio; los racimos y cambios de vocabulario sugieren contextos

EventStorming: encontrar fronteras con el negocio

Enfoque bottom-up — las fronteras *emergen* del comportamiento real. Taller donde negocio y desarrollo mapean el dominio en una pared, en orden temporal, con post-its de colores:

- **Evento de dominio** (*naranja*) — algo relevante que pasó ("Pedido confirmado")
- **Comando** (*azul*) — lo que lo provoca ("Confirmar pedido")
- **Agregado** (*amarillo*) — sobre qué recae el comando
- **Política / proceso** (*lila*) — "cuando pasa X, automáticamente Y"

Es la vía más rápida de encontrar fronteras *con* el negocio en la sala. **Cómo emergen exactamente** —racimos, vocabulario, agregados y políticas— en las dos slides siguientes.

De los racimos a las fronteras: cómo emergen

Colocados **en orden temporal**, los eventos se apilatan solos. Donde el racimo se corta (||) hay una frontera:

Carrito creado	Pago autorizado	Envío despachado
Pedido confirmado	Factura emitida	Entrega confirmada
_____	_____	_____
objetivo: VENDER	objetivo: COBRAR	objetivo: ENTREGAR
vocab: carrito·pedido	vocab: pago·factura	vocab: envío·paquete
= Contexto Ventas	= Contexto Pagos	= Contexto Logística

Por qué ahí corta una frontera: en la || saltan a la vez el **objetivo** del negocio, el **vocabulario** (mismo dato, otro nombre y campos) y el **responsable** (Ventas → Finanzas → Logística).

Enriquecer la pared: los demás post-its

Los **eventos encuentran** las fronteras; los demás post-its, añadidos por capas, las **validan y tipifican**:

Post-it	Pregunta que responde	Efecto en la frontera
Comando + actor <i>(azul)</i>	¿quién lo dispara y cómo lo llama?	confirma la línea (vocabulario, rol)
Agregado <i>(amarillo)</i>	¿qué cambia junto?	cohesión <i>dentro</i> — mismo nombre como 2 agregados = parte el racimo
Política <i>(lila)</i>	¿quién reacciona a quién?	contrato <i>entre</i> contextos (evento async)

Terminas no solo con los contextos, sino con los **contratos entre ellos** (las políticas) — justo lo que necesitas para planificar la migración.

Capacidad de negocio vs. subdominio

Dos heurísticas **top-down** para proponer contextos —se *deducen* del propósito y la estructura, y complementan el EventStorming **bottom-up**:

- **Por capacidad de negocio:** ¿qué *hace* la empresa? (gestionar pedidos, facturar). Estable, de fuera adentro. Riesgo: copiar el organigrama actual.
- **Por subdominio:** ¿de qué se *compone* el dominio?
 - **Core** — lo que te diferencia. Mejor equipo, mejor diseño.
 - **Supporting** — necesario, no diferencial; a medida porque no hay alternativa.
 - **Generic** — lo resuelve una solución comprada (contabilidad, login, email).

core / supporting / generic = cómo **repartir el esfuerzo**: modelado fino al core; lo genérico a menudo **ni se migra — se reemplaza** por SaaS.

Un servicio $\approx$ un bounded context

Un microservicio **no debe ser más pequeño que un bounded context**; de partida, **un contexto $\approx$ un servicio**.

Por qué funciona: dentro de un contexto la cohesión es alta (cambia junto $\rightarrow$ vive junto); entre contextos el acoplamiento es bajo (pocas interacciones, formalizables como contratos). Justo lo que pide el despliegue independiente.

Por qué fracasa cortar por capa o entidad: "añadir descuentos por volumen" atraviesa capas y entidades $\rightarrow$ toca N servicios $\rightarrow$ monolito distribuido.

Ejercicios 3.1 y 3.2

3.1 · Detectar bounded contexts por el lenguaje

3.2 · Core, supporting o generic

(ver cuaderno de ejercicios)

2. Encontrar los límites

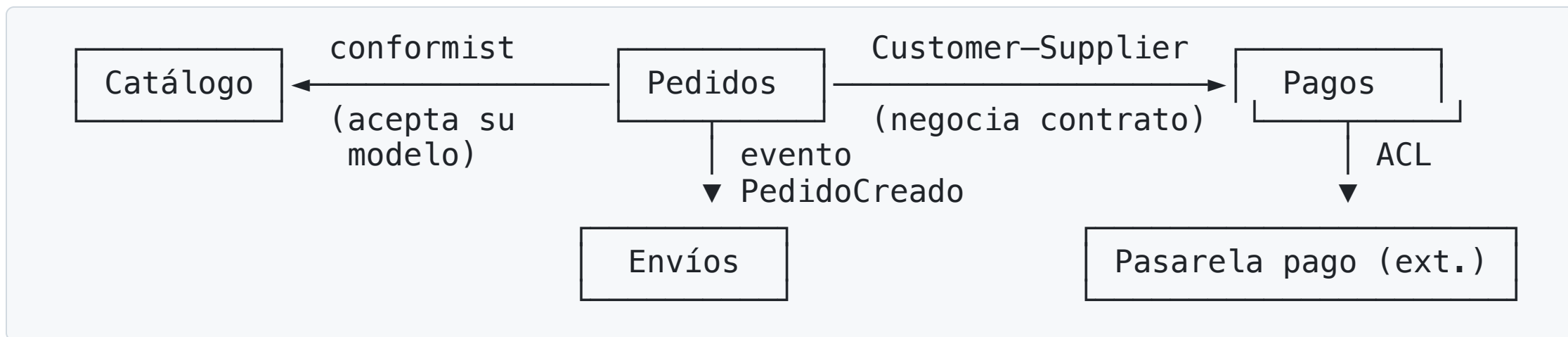
Context maps

El **plano de los contextos y sus relaciones**. Saber cuáles son no basta: hay que saber cómo se influyen (eso fija contratos y poder de negociación).

Relación	Significado
Customer–Supplier	El proveedor tiene en cuenta al consumidor (hay negociación)
Conformist	El consumidor acepta el modelo del proveedor tal cual
Anticorruption Layer	El consumidor se protege traduciendo (Sesión 2)
Shared Kernel	Comparten parte del modelo (acoplamiento fuerte — con moderación)
Open Host / Published Language	API/formato estable y documentado para muchos
Integración por eventos (pub/sub)	El consumidor reacciona a eventos publicados, sin llamada directa (la <i>Política</i> de EventStorming)
Separate Ways	No se integran: duplicar es más barato que coordinar

Un context map de ejemplo

Las mismas relaciones, dibujadas sobre los contextos de un e-commerce:



Saber los contextos no basta: el mapa fija **quién depende de quién**, dónde pones un **ACL** y quién tiene **poder de negociación** sobre el contrato.

Granularidad: ¿demasiado pequeño?

Señales de haber cortado **demasiado fino**:

- **Cambios en cascada**: una función media toca varios servicios
- **Charla excesiva**: dos servicios siempre juntos en las trazas
- **Transacciones partidas**: necesitas consistencia fuerte *entre* servicios
- **Datos siameses**: dos servicios leen/escriben las mismas tablas
- Más servicios que desarrolladores

Ante la duda, corta más grueso. Dividir un servicio grande es barato; **fusionar** dos pequeños (unir modelos, datos y contratos publicados) es caro.

3. Descomposición de datos

El problema: la BD compartida

Extrae todo el código que quieras: mientras **compartan base de datos, siguen acoplados**.

Usaremos este esquema de ejemplo el resto de la sección:

BD única del monolito (e-commerce)

clientes · pedidos · pagos · envios · catalogo · paises · monedas

todos los módulos leen y escriben cualquier tabla

- El **esquema es un contrato implícito**: nadie cambia una tabla sin romper a los demás → adiós despliegue independiente
- La lógica acaba en el sitio equivocado (se reimplementa y diverge en cada consumidor)
- No puedes cambiar de tecnología de almacenamiento por servicio
- El rendimiento de uno afecta a todos (bloqueos, pools, picos)

Objetivo: base de datos por servicio

Cada servicio es el **único** que accede a sus datos; los demás pasan por su **API**.

- Restaura el **despliegue independiente** (cada uno cambia su esquema)
- La lógica de negocio vive en **un** sitio (su dueño)
- Cada servicio elige su **tecnología** de almacenamiento
- Aísle el rendimiento

Es la parte **más difícil y más aplazada** de toda migración.

Dividir los datos: propiedad primero

1. **Mapear propiedad:** por cada tabla, ¿qué contexto la *escribe*? (las que escribe todo el mundo = alarma)
2. **Separar el esquema lógicamente:** esquemas/usuarios distintos en la misma instancia, sin cruces. Barato, revela todos los acoplamientos
3. **Romper los accesos cruzados** uno a uno → API del dueño · copia local por eventos · mover el dato de dueño
4. **Separar físicamente** (instancia propia) cuando ya no hay cruces

Romper un acceso cruzado: tres opciones

Cuando un contexto **lee/escribe una tabla de otro**, cortas ese hilo con una de tres:

Opción	Cuándo usarla	Efecto
1. API del dueño <i>(síncrono)</i>	necesitas el dato fresco al momento (p.ej. stock antes de vender)	dato exacto, pero acopla en runtime : latencia y dependes de que el dueño esté vivo
2. Copia local por eventos <i>(asíncrono)</i>	toleras desfase (consistencia eventual; p.ej. nombre del cliente en Envíos)	desacopla en runtime (sigues vivo si el dueño cae); duplicas el dato
3. Mover el dato de dueño	el dato pertenece a otro contexto (el reparto del paso 1 estaba mal)	elimina la relación en vez de gestionarla — la más limpia cuando aplica

¿El dato es de otro contexto? → **muévelo**. Si no: ¿lo necesitas fresco? → **API**; ¿toleras desfase? → **eventos** *(la opción por defecto: es la única que rompe el acoplamiento en tiempo de ejecución)*.

Casos típicos difíciles

Las tres opciones rompen un acceso *simple*. Sobre el esquema de antes, tres **formas de datos** piden un matiz extra:

- **Datos de referencia que todos leen** (`países` , monedas): API o eventos serían exagerados para algo casi inmutable → **duplica una copia** en cada servicio (o un mini-servicio de referencia si cambia)
- **Una tabla que esconde dos dueños** (`pedidos` mezcla columnas de pedido y de envío): no es elegir entre las tres → **pártela por columnas**; la clave (`pedido_id`) queda como referencia entre Pedidos y Envíos
- **Informes que cruzan dominios** (Clientes × Pedidos): ya no hay JOIN entre BDs → **composición en el llamante**, o **vista materializada por eventos** —la copia local del slide anterior aplicada a consultas (§4.5)

Partir una tabla en la práctica

`pedidos` esconde **dos dueños**: datos de pedido (→ Pedidos) y de envío (→ Envíos).

ANTES – una tabla, dos dueños

`pedidos`

`id` (PK)

`cliente_id`

`total`

`direccion_envio`

`transportista`

`estado_envio`



DESPUÉS – dos servicios

`pedidos-svc`

`id` (PK)

`cliente_id`

`total`

`envios-svc`

`id` (PK)

`pedido_id` (ref→)

`direccion`

`transportista`

`estado_envio`

`pedido_id` deja de ser una FK con integridad de BD y pasa a ser una **referencia entre servicios**: la valida la app, puede quedar "colgada" un instante, y se sincroniza por **eventos** (`PedidoCreado` → Envíos crea su fila).

El coste de los datos distribuidos

Sin anestesia: al partir los datos **se pierde** lo que la BD única regalaba.

Se pierde	Lo sustituye (peor, más trabajo)
Transacciones ACID entre dominios	Sagas + consistencia eventual
JOIN entre cualquier par de tablas	Composición por API / vistas materializadas
Integridad referencial (FK) global	Validación en app; tolerar referencias rotas
Un único lugar donde mirar	Datos repartidos; catálogo/linaje

Por eso los **límites importan tanto**: buen corte → transacciones y joins entre servicios raros. Los invariantes con consistencia fuerte van **dentro de un servicio** (un *agregado*).

Ejercicio 3.3

Partir la base de datos compartida

(ver cuaderno de ejercicios)

4. Consistencia sin transacciones distribuidas

Agregados: la unidad de consistencia

El único trozo de DDD **táctico** que la migración necesita.

Un **agregado** = grupo de objetos que cambian juntos bajo una **regla de negocio** (invariante) y se tratan como **una sola unidad**. Tiene una **raíz**; desde fuera solo se habla con la raíz.

Ej.: **Pedido + sus líneas** = un agregado. El invariante "el total no supera el límite de crédito" obliga a validarlas juntas → no se tocan las líneas por separado.

Regla de oro de consistencia: un invariante que exige **ACID** debe caber dentro de un agregado → dentro de **un** servicio. Lo que cruza agregados/servicios va con consistencia **eventual** (sagas).

Por qué no usamos 2PC

La transacción distribuida clásica (*two-phase commit*): un coordinador pide votar a todos y luego confirma. En microservicios **se evita**:

- **Bloquea recursos** en todos los participantes mientras dura → mata el rendimiento
- El **coordinador** es un punto único de fallo, con estados "en duda" horribles
- **Acopla la disponibilidad** de todos (avanza al ritmo del más lento/caído)
- Muchas tecnologías (colas, NoSQL, HTTP) **no lo soportan**

Consistencia eventual

Aceptar estados intermedios visibles y garantizar que el sistema **converge**: *"el pedido está creado pero el stock aún no descontado... durante 200 ms."*

No es (solo) un problema técnico, es una conversación de negocio. Casi todos los procesos reales *ya son* eventualmente consistentes (la tienda física vende y el stock central se entera por la noche).

Preguntas correctas — **las responde negocio**: ¿qué desfase es aceptable aquí? ¿qué hacemos si al final no se puede completar?

El patrón Saga

Una **saga** = secuencia de transacciones **locales** (cada una ACID en su servicio) que juntas implementan un proceso. Si un paso falla, no hay rollback global: se ejecutan **compensaciones** (acciones de negocio que deshacen).

"Realizar pedido": 1) Pedidos crea → 2) Stock reserva → 3) Pagos cobra → 4) Pedidos confirma. Si el cobro (3) falla → compensar 2 (liberar reserva) y 1 (marcar pedido **cancelado**, no borrado).

La compensación es una **acción de negocio** con su semántica, no un "deshacer" mágico.

Saga: camino feliz y compensación

CAMINO FELIZ

1. Pedidos: crear pedido
 2. Stock: reservar
 3. Pagos: cobrar
 4. Pedidos: confirmar
- = PEDIDO CONFIRMADO

FALLO EN EL COBRO (paso 3)

1. Pedidos: crear pedido
 2. Stock: reservar
 3. Pagos: cobrar $\longrightarrow$ x rechazado
 - ⌘ Stock: liberar reserva (compensa 2)
 - ⌘ Pedidos: marcar CANCELADO (compensa 1)
- = PEDIDO CANCELADO (no borrado)

No hay rollback global: cada compensación es una **acción de negocio** ejecutada hacia atrás. Por eso el **orden** de los pasos es una decisión (Ejercicio 3.4): deja lo más caro de compensar para el final.

Orquestación vs. coreografía

Orquestación — un coordinador central llama a cada paso y decide compensar.

- ✓ proceso explícito en un sitio; fácil de razonar, depurar, modificar
- ⚠ concentra acoplamiento (lo conoce todo); puede volverse un "dios"

Coreografía — sin coordinador; cada servicio reacciona a eventos y emite los suyos.

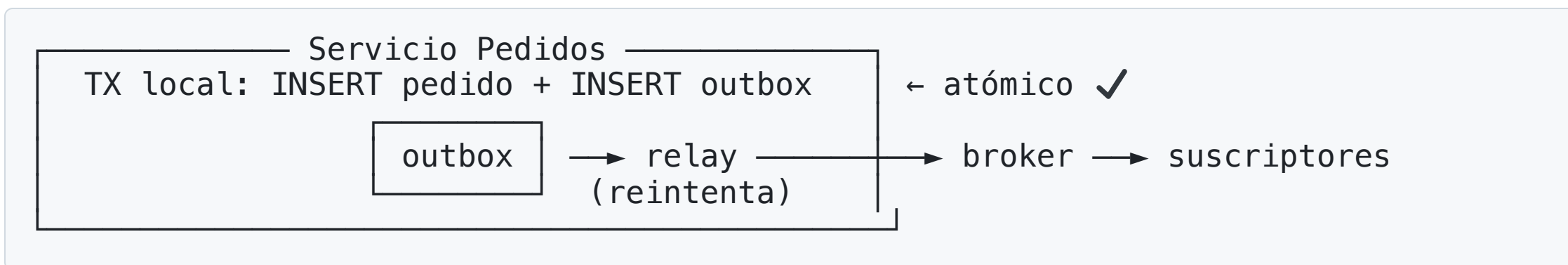
- ✓ acoplamiento mínimo, servicios autónomos, encaja con eventos
- ⚠ el proceso global **no está escrito** en ningún sitio: emerge. "¿Dónde está el pedido 4711?"

Procesos cortos y estables → coreografía. Largos, con ramas/compensaciones o que exigen visibilidad → orquestación. Mezclar vale.

La idea del Outbox

Un servicio debe guardar el cambio en su BD **y** publicar un evento — pero son **dos sistemas, sin transacción común**, y cada mitad puede fallar sola:

- **Guarda y no publica** (cae tras el commit) → **evento perdido**: pasó, pero nadie se entera
- **Publica y no guarda** (rollback) → **evento fantasma**: otros reaccionan a algo que no pasó



El **relay** publica leyendo la outbox: por *polling* de la tabla, o por **CDC** sobre el log de la BD (p. ej. Debezium).

Outbox → al-menos-una-vez → idempotencia

El relay puede **reenviar** tras un fallo → la entrega es **al-menos-una-vez**.

→ Los consumidores deben ser **idempotentes**: procesar el mismo evento dos veces sin daño.

- **Deduplicar por** `event_id` (marca de procesado en la misma TX que el efecto), o
- **Idempotencia natural** por clave de negocio (`INSERT ... ON CONFLICT DO NOTHING`)

Outbox (productor) + idempotencia (consumidor) = el fundamento de fiabilidad de casi toda arquitectura de eventos.

Consultas e informes entre servicios

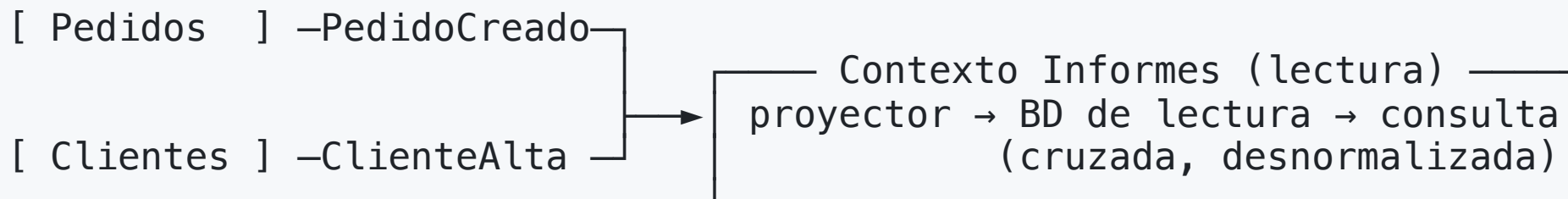
Sin JOINS globales, ¿"los 10 clientes con más pedidos este mes"?

1. **Composición de APIs:** el llamante consulta varios servicios y combina. Bien para pocas entidades; mal para agregaciones masivas
2. **Vista materializada / CQRS** (*Command Query Responsibility Segregation: separar el modelo de escritura del de lectura*): un componente se **suscribe a los eventos** y mantiene una BD de lectura ya cruzada. Eventualmente consistente (suele bastar para informes)
3. **Plataforma analítica:** informes pesados → data warehouse alimentado por eventos, no contra las BDs operacionales

Las consultas que cruzan dominios son **otro consumidor de eventos**, no una excusa para volver a la BD compartida.

CQRS: vista de lectura por eventos

"Los 10 clientes con más pedidos del mes" cruza dos servicios. En vez de un JOIN, un componente mantiene una tabla ya cruzada, **solo de lectura**:



- **Escritura y lectura** usan modelos distintos → eso es **CQRS**
- Es **eventualmente consistente** (desfase de segundos; suele bastar para informes)
- Si se corrompe, se **reconstruye** reproduciendo los eventos

Reusa los **mismos eventos** de las sagas: el reporting es otro consumidor de eventos, no una excusa para volver a la BD compartida.

Ejercicio 3.4 + 3.5

Diseñar una saga

¿Dónde falla esto? (outbox e idempotencia)

(ver cuaderno de ejercicios)

Resumen de la sesión

- El corte sigue al **negocio**: bounded contexts con lenguaje ubicuo; un término con dos significados = una frontera
- **Context maps** para gobernar relaciones; **core/supporting/generic** para repartir esfuerzo (lo genérico se compra)
- Granularidad: ante la duda, **cortar grueso** (dividir es barato, fusionar es caro)
- Sin **BD por servicio** no hay despliegue independiente: propiedad → esquemas → eliminar cruces → separar
- Sin transacciones distribuidas: **consistencia eventual** con semántica de negocio, **sagas** con compensaciones, **outbox + idempotencia**

¿Preguntas?

Próxima sesión: Comunicación, resiliencia y organización

Sync/async · Circuit breaker · Observabilidad · Conway · Team Topologies